Loyalty Program Synthetic Dataset Generation Framework

Documentation and Design Specification

Prepared by: Darlyn Baybayon and Angela Taningco

Contents

Introduction **2**

Use Case Description **2**

Dataset Overview **3**
- Schema . . . 3
- Entity Tables . . . 3
- Relationships . . . 5

Data Generation Pipeline **7**
- Generate Members Table . . . 7
- Transaction Simulation . . . 8

Output Datasets **17**

Setup and Reproducibility **19**
- Environment Requirements . . . 19
- Dependencies . . . 19
- Execution Instructions . . . 19

Limitations and Future Improvements **20**

Introduction

Loyalty programs are widely used by businesses to encourage repeat purchases, strengthen customer relationships, and generate behavioral data. Analyzing such programs requires detailed and structured datasets that capture customer activity, transactions, and reward interactions.

This project presents a synthetic data generation system that simulates a loyalty program for a coffee shop. The system models key components such as members, transactions, points accumulation, reward redemption, and tier progression. By incorporating realistic statistical distributions and business rules, the generated dataset aims to reflect typical customer behavior and program dynamics.

Use Case Description

The synthetic loyalty program dataset is designed to support analytics and decision-making for a retail business, specifically a coffee shop. It enables the simulation of customer behavior and interaction with a points-based rewards system.

The primary use case is to analyze how customers earn, redeem, and lose points over time, and how these activities influence purchasing patterns and tier progression. The dataset allows analysts to explore key business questions such as:

- How frequently do customers make purchases and earn points?
- What proportion of points are redeemed versus expired (breakage)?
- How do customers transition between loyalty tiers?
- Which customer segments contribute the most revenue and engagement?

Additionally, the dataset may be used to develop business intelligence dashboards, predictive models, and customer segmentation strategies. It provides a controlled environment for assessing loyalty program performance and experimenting with potential improvements.

Dataset Overview

Schema

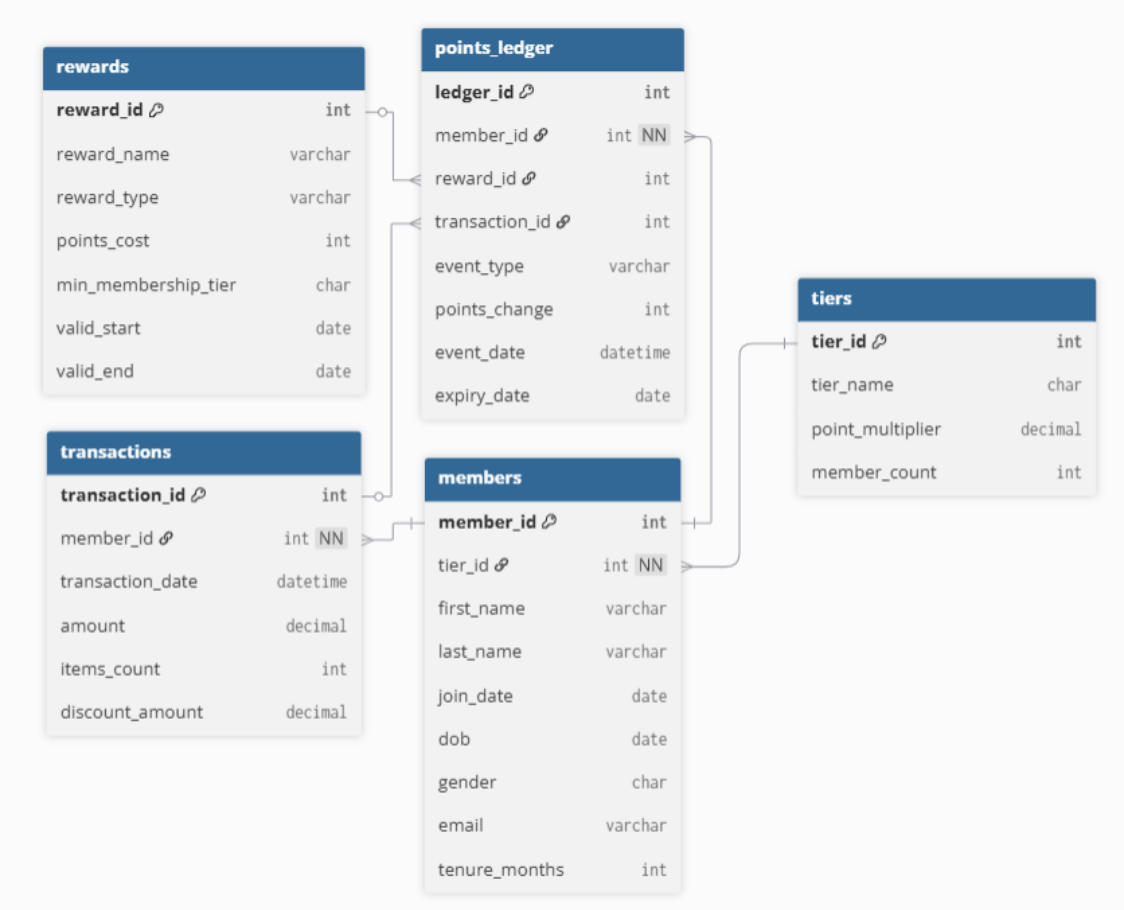


Figure 1: Entity Relationship Diagram

Entity Tables

Members

The members table stores demographic and profile information of each customer in the loyalty program. It serves as the central entity that links to transactions and points activity.

Field	Type	Description
member_id (PK)	INT	Unique identifier per member
first_name	VARCHAR	Member's first name
last_name	VARCHAR	Member's last name
date_of_birth	DATE	Member's birth date
join_date	DATE	Date the member joined the program
age	INT	Member's age
email	VARCHAR	Member's email address
gender	CHAR	Member's gender
current_tier (FK)	INT	References current loyalty tier of Memvber
tenure_months	INT	Length (in months) of member's tenure in the program

Table 1: Members Entity Table

Transactions

The transactions table contains all purchasing activities and records details such as transaction date, amount spent, number of items, discount amount.

Field	Type	Description
transaction_id (PK)	INT	Unique identifier per transaction
member_id (FK)	INT	References member who made the transaction
transaction_date	DATETIME	Date the transaction transpired
amount	DEC	Total amount spent in the transaction
items_count	INT	No. of items in the transaction
discount_amount	DEC	Amount discounted from the transaction

Table 2: Transactions Entity Table

Tiers

The tiers table contains information about the different tiers (Bronze, Silver, Gold) of the loyalty program including the member count and point multiplier to be used for calculating the earned points.

Field	Type	Description
tier_id (PK)	INT	Unique identifier per tier
tier_name	CHAR	Name of tier
member_count	INT	No. of members in the tier
points_multiplier	DEC	Points multiplier per tier

Table 3: Tiers Entity Table

Rewards

The rewards table stores the catalog of all rewards in the loyalty program along with their availability periods and point cost.

Field	Type	Description
reward_id (PK)	INT	Unique identifier per reward
reward_name	VARCHAR	Name of reward
reward_type	CHAR	Type of reward (e.g. Food, Merch)
min_membership_tier	CHAR	Minimum tier required for viability
valid_start	DATE	Starting date of validity
valid_end	DATE	Ending date of validity
points_cost	INT	Cost of redemption

Table 4: Rewards Entity Table

Points Ledger

The points ledger table records all point-related events for each member. This includes earning events from transactions, burning through reward redemptions, and expiry, providing a complete log of all changes in a member's point balance.

Field	Type	Description
ledger_id (PK)	INT	Unique identifier per ledger
member_id (FK)	INT	References member associated with the event
transaction_id (FK)	INT	References the related transaction (if applicable)
reward_id (FK)	INT	References reward redeemed (if applicable)
event_date	DATETIME	Date when event occurred
event_type	CHAR	Type of event (EARN, BURN, EXPIRY)
points_change	INT	No. of points added or deducted
expiry_date	DATE	Date of point expiry

Table 5: Points Ledger Entity Table

Relationships

The relationships between the entities define how data flows within the loyalty program system.

An **optional one-to-many relationship (1:0-N)** exists between **Members and Transactions**, where a single member can have multiple transactions, but each transaction is associated with only one member. This also means members may have no associated transactions records at all (inactive accounts with no purchases).

Similarly, **Members and Points Ledger** follow an **optional one-to-many relationship (1:0-N)**, as each member can have multiple points-related events recorded over time but a log in the Points Ledger table is associated strictly to one member. Furthermore, since members with no purchases exist, members may also not have any records in the point ledger as they have not earned points from transactions.

The relationship between **Transactions and Points Ledger** is **optional one-to-one (0–1:1)**. This means that a transaction strictly has one associated points ledger entry (i.e. points earned), but not all points-related events involve a transaction (i.e. rewards redemption).

Likewise, Rewards and Points Ledger have an **optional one-to-one (0–1:1) relationship**. A

points ledger entry related to redemption may reference a reward, but not all ledger entries involve rewards, such as earning or expiry events.

Finally, there is a **many-to-one relationship (N:1)** between Members and Tiers. Multiple members can belong to the same tier, but each member is assigned to only one tier at any given time.

Data Generation Pipeline

The data generation pipeline simulates the lifecycle of a loyalty program by sequentially generating members, transactions, and points-related events. Each step builds on the previous one to ensure consistency across tables and adherence to business rules.

Pipeline Structure

The data generation process follows a sequential workflow:

1. Member data generation
2. Transaction simulation
3. Tier assignment
4. Rewards catalog generation
5. Points ledger construction

Generate Members Table

The pipeline begins by generating a synthetic member base using the `generate_member_data()` function. This function generates the base `members` dataset, including demographic attributes, join dates, and identifiers.

Parameter	Description
start_date	Start date of the data generation period
end_date	End date of the data generation period; defaults to today if not provided
initial_members	Initial number of members at the start date
growth_rate	Rate at which the member base increases over time
seed	Random seed for reproducibility

Table 6: `generate_member_data()` Parameters

- **Population Growth Model**
  The number of members grows over time according to:

$$N_t = N_0(1+g)^t$$

  where N_0 is the initial number of members, g is the monthly growth rate, and t is time in months (computed from `start_date` and `end_date`). This model assumes that member acquisition grows proportionally with the existing base, reflecting increased shop popularity over time. While it does not explicitly model irregular fluctuations such as sudden demand shocks, this simplification allows for a stable and scalable representation of member expansion.

- **Age Distribution**
  Age is sampled from a mixture distribution:

$$\text{Age} \sim \begin{cases} \mathcal{N}(28, 5^2), & \text{with probability } 0.7 \\ \mathcal{N}(45, 8^2), & \text{with probability } 0.3 \end{cases}$$

to simulate realistic customer demographic segmentation. Values are clipped to the interval $[16, 65]$. Based on several local studies, most people who frequent coffee shops are young adults—university students and young professionals [1] [2].

- **Join Date Distribution**
  Join dates are sampled from the full date range using a sinusoidal weighting function:

$$p_t = \frac{1.5 + \sin(t)}{\sum_{k=1}^{T}(1.5 + \sin(k))}$$

  where t is the index of a specific day in the date range, T is the total number of days in the simulation period, p_t is the probability of selecting day t, and k is the index used to normalize probabilities across all days.

  Dates are then drawn as:

$$d_i \text{ Categorical}(p_1, p_2, ..., p_T)$$

  where d_i is the assigned join date of member i.
  This introduces periodic variation in member sign-ups, allowing certain periods to have higher or lower acquisition rates.

- **Gender Distribution**
  Gender is assigned probabilistically per member as:

$$G_i \text{ Categorical}(0.30, 0.50, 0.20)$$

  where G_i is the gender of member i, corresponding to categories Male, Female, and Other, respectovely. This distribution reflects reflects the observed dominance of female customers in coffee shops according to local studies [2]. Gender is also used as an input for generating synthetic names using the Faker Python library and assembling email addresses, ensuring consistency between demographic attributes and identity fields.

Transaction Simulation

The member data is passed onto the `generate_transaction_data(members, seed, end_date)` function to simulate purchase behavior for each member.

Parameter	Description
members	Generated members dataset used as input for transaction simulation
seed	Random seed for reproducibility
end_date	End date of the transaction period; defaults to today if not provided

Table 7: `generate_transaction_data()` Parameters

- **Transaction Frequency**
  For each member, the expected number of transactions is based on age and tenure. A base rate is first assigned by age group:

$$\lambda_{\text{base}} = \begin{cases} 20, & \text{age} < 30 \\ 15, & 30 \leq \text{age} < 45 \\ 5, & \text{age} \geq 45 \end{cases}$$

  This is adjusted by membership tenure:

$$\lambda_i = \lambda_{\text{base}} \left(\frac{\text{tenure}_i}{12} \right)^e$$

  where $e \sim U(0.5, 1.2)$. The number of candidate transactions is then sampled as:

$$T_i \sim \text{Poisson}(\max(\lambda_i, 1))$$

  Following the assumption that younger customers frequent coffee shops more, this simulation reflects the younger segment of members are more likely to have more transactions while also accounting for each member's tenure length.

- **Transaction Dates and Time**
  For each candidate transaction, the date d is sampled uniformly between the member's join date and the simulation end date:

$$d_{ij} \sim U(\text{join_date}_i, \text{end_date})$$

  where i indexes the member and j indexes the transaction of member i.
  A transaction time is then added using store operating hours, with hours sampled from 10:00 to 21:59. This ensures that transactions occur only after the member joined and within realistic business hours.

- **Transactional Seasonality**
  Transaction seasonality is applied after candidate transactions are generated. Each candidate transaction is retained based on weekly and monthly factors. The weekly retention probability is:

$$p_{\text{week}} = \begin{cases} U(0.6, 1.0), & d \geq 4 \\ U(0.2, 0.5), & d < 4 \end{cases}$$

  where d is the day of the week.

  A monthly seasonal multiplier is also applied:

$$m_{\text{season}} \sim U(a_m, b_m)$$

  where (a_m, b_m) depends on the transaction month.

  The final retention probability is:

$$p_{\text{final}} = \min(p_{\text{week}} \cdot m_{\text{season}}, 1)$$

A transaction is retained if:

$$u \leq p_{\text{final}}, \quad u \sim U(0,1)$$

- **Item Count and Price**
  The number of items Q purchased by member i in each retained transaction j is generated as:

  $$Q_{ij} \sim \text{Poisson}(2) + 1$$

  where the additional 1 ensures at least one item per transaction. Item prices P are sampled from a lognormal distribution:

  $$P_{ij} \sim \text{LogNormal}(4.8, 0.3^2)$$

- **Transaction Amount**
  The initial transaction amount is computed as:

  $$A_{ij} = Q_{ij} P_{ij}$$

  Prices and amounts are rounded to realistic retail increments.

- **Discount Application**
  Discounts D are applied probabilistically using:

  $$D_{ij} \sim \text{Bernoulli}(0.30)$$

  If a discount is applied, the discount rate is sampled as:

  $$r_{ij} \sim \text{Beta}(2, 8)$$

  The final transaction amount is:

  $$A^*_{ij} = A_{ij} - (A_{ij} r_{ij})$$

  This produces mostly small discounts while allowing occasional larger discounts.

Tier Assignment

In this step, the tiers are defined and each member is assigned a tier based on their level of engagement using the `generate_tiers_assignment()` function

Parameter	Description
members	Members dataset containing member profiles and join dates
transactions	Transactions dataset used to compute member activity metrics
end_date	Cutoff date for tier assignment; defaults to today if not provided

Table 8: `generate_tiers_assignment()` Parameters

- **Tier Assignment**
  Member tiers are assigned based on recent transaction activity. The function uses the simulation end date as the cutoff date and evaluates member behavior over three time windows: the last 12 months, last 6 months, and last 3 months.

- **Frequency and Monetary Metrics**
  For each member, the system computes transaction frequency and total spending over the last 12 months:

$$F_i = \sum_{j=1}^{n_i} 1$$

$$M_i = \sum_{j=1}^{n_i} A_{ij}$$

  where:

  - F_i is the number of transactions made by member i in the last 12 months.
  - M_i is the total amount spent by member i in the last 12 months.
  - A_{ij} is the amount of transaction j made by member i.

  These metrics represent how often a member transacts and how much value they contribute to the program.

- **Activity Streak Metric**
  The system also measures recent engagement using the number of active months in the last 6 months:

$$S_i = \sum_{m=1}^{6} I(T_{im} > 0)$$

  where:

  - S_i is the number of months in which member i had at least one transaction.
  - T_{im} is the number of transactions of member i in month m.
  - $I(\cdot)$ is an indicator function equal to 1 if the condition is true and 0 otherwise.

  This captures consistency of engagement, not just total spending.

- **Min-Max Normalization**
  Since frequency, monetary value, and active months have different scales, each metric is normalized to a 0–1 range:

$$X_i^{\text{norm}} = \frac{X_i - X_{\min}}{X_{\max} - X_{\min} + 10^{-6}}$$

  where:

  - X_i represents one of the metrics F_i, M_i, or S_i.

- $X_{\min}$ and $X_{\max}$ are the minimum and maximum values of that metric.
- 10^{-6} is added to avoid division by zero.

This allows the three metrics to be combined fairly despite having different units.

- **Composite Tier Score**
  A weighted score is computed for each member:

$$\text{tier_score}_i = 0.5F_i^{\text{norm}} + 0.4M_i^{\text{norm}} + 0.1S_i^{\text{norm}}$$

  where:

  - F_i^{norm} is the normalized transaction frequency.
  - M_i^{norm} is the normalized monetary value.
  - S_i^{norm} is the normalized active-month count.

  The weights prioritize transaction frequency and spending, while still giving a smaller contribution to consistency of activity.

- **Percentile-Based Tier Classification**
  Members are assigned tiers based on their composite score percentiles:

$$\text{Tier}_i = \begin{cases} \text{Gold}, & \text{tier_score}_i \geq Q_{0.90} \\ \text{Silver}, & Q_{0.70} \leq \text{tier_score}_i < Q_{0.90} \\ \text{Bronze}, & \text{tier_score}_i < Q_{0.70} \end{cases}$$

  where:

  - $Q_{0.90}$ is the 90th percentile of the tier score distribution.
  - $Q_{0.70}$ is the 70th percentile of the tier score distribution.

  This creates an approximate 70% Bronze, 20% Silver, and 10% Gold distribution, while allowing the cutoffs to adapt to the generated data.

- **Inactivity-Based Demotion**
  After initial tier assignment, members with no transactions in the last 3 months are demoted by one tier:

$$\text{Tier}_i = \begin{cases} \text{Silver}, & \text{Tier}_i = \text{Gold and } R_i = 0 \\ \text{Bronze}, & \text{Tier}_i = \text{Silver and } R_i = 0 \\ \text{Tier}_i, & \text{otherwise} \end{cases}$$

  where R_i is the number of transactions made by member i in the last 3 months. This rule ensures that high tiers reflect not only past value but also recent activity.

- **Tier Table Generation**
  After final tier assignments, a tier summary table is created with the number of members per tier and the corresponding point multiplier:

$$m_i = \begin{cases} 1.0, & \text{Bronze} \\ 1.2, & \text{Silver} \\ 1.5, & \text{Gold} \end{cases}$$

where m_i is the point multiplier applied to member i when earning points. This links the tier assignment process to the points ledger, since higher-tier members earn points at a faster rate.

Rewards Generation

The `rewards` dataset, including reward types, point costs, and availability periods using the `generate_rewards()` function.

Parameter	Description
start_date	Start date used to define reward availability periods
end_date	End date used to define reward availability periods
seed	Random seed for reproducibility

Table 9: `generate_rewards()` Parameters

- **Reward Pool**
  Rewards are generated from a predefined pool and made available over rotating time windows. Each reward cycle spans:

  $$L_c \sim U(60, 120)$$

  where L_c is the cycle length in days. For each cycle c, a subset of rewards is selected:

  $$R_c \subseteq \mathcal{R}, \quad |R_c| \sim U\{4, 6\}$$

  where $\mathcal{R}$ is the full reward pool.

- **Reward Cost**
  Each reward $r \in R_c$ is assigned a points cost based on its base value with random variation:

  $$C_r = \max(100,\ C_r^{\text{base}} + \delta), \quad \delta \in \{-50, 0, 50, 100\}$$

- **Reward Validity**

  Each reward is assigned a valid interval:

  $$[\text{start}_r, \text{end}_r] = [\text{cycle}_c^{\text{start}}, \text{cycle}_c^{\text{end}}]$$

- **Birthday Reward**
  Additionally, a special birthday reward is defined with zero cost:

  $$C_{\text{birthday}} = 0$$

Points Ledger Generation

With the updated member data, transactions, and rewards catalog, the points ledger, which tracks all point-telated events, is generated by the `generate_points_ledger()` function. Points are earned through purchases and burned through reward redemption or expiry.

Parameter	Description
members	Members dataset containing member tiers, join dates, and birthdays
transactions	Transactions dataset used to create point-earning events
rewards	Rewards catalog used for redemption events
birthday_reward	Special zero-cost reward assigned during member birthdays
tiers	Tier table containing point multipliers for each membership tier
start_date	Start date of the ledger generation period
end_date	End date of the ledger generation period
expiry_days	Number of days before earned points expire; defaults to 365
burn_lambda	Expected redemption rate used in the Poisson redemption model
seed	Random seed for reproducibility

Table 10: `generate_points_ledger()` Parameters

- **Earn Events**
  For each transaction, points are computed using the transaction amount and the member's current tier multiplier:

  $$P_{\text{earn},ij} = \text{round}(A_{ij} \cdot m_i)$$

  where:

  - A_{ij} is the amount of transaction j for member i.
  - m_i is the point multiplier associated with the current tier of member i.
  - $P_{\text{earn},ij}$ is the number of points earned from the transaction.

  The earned points are stored as positive `EARN` events with `TRANSACTION` as the point type.

- **Expiry Events**
  Each transaction-based earn event is assigned a rolling expiration date based on the value of `expiry_days`:

  $$d_{\text{expire},ij} = d_{\text{transaction},ij} + \texttt{expiry_days}$$

  where:

  - $d_{\text{transaction},ij}$ is the transaction date of transaction j for member i.
  - $d_{\text{expire},ij}$ is the corresponding expiration date.
  - `expiry_days` is the number of days before earned points expire.

If $d_{\text{expire},ij}$ falls within the simulation period, an expiration event is added:

$$P_{\text{expire},ij} = -P_{\text{earn},ij}$$

This records the removal of unused earned points after the validity period.

- **Redemption Events**
  For each member with a positive point balance, the number of redemption attempts is sampled using a Poisson distribution:

$$R_i \sim \text{Poisson}(\lambda_{\text{burn}})$$

  where:

  - R_i is the number of redemption attempts for member i.
  - λ_{burn} is the expected redemption rate controlled by `burn_lambda`.

  A reward is considered eligible if it satisfies the following conditions:

$$C_r \leq B_i$$

$$\text{tier}_r \leq \text{tier}_i$$

$$[d_{\text{start},r}, d_{\text{end},r}] \cap [d_{\text{join},i}, d_{\text{end}}] \neq \emptyset$$

  where:

  - C_r is the point cost of reward r.
  - B_i is the current point balance of member i.
  - tier_r is the minimum tier required for reward r.
  - tier_i is the current tier of member i.
  - $d_{\text{start},r}$ and $d_{\text{end},r}$ are the validity dates of reward r.
  - $d_{\text{join},i}$ is the join date of member i.
  - d_{end} is the simulation end date.

  Once a reward is selected, the redemption event reduces the member's balance by the reward cost:

$$P_{\text{burn},ir} = -C_r$$

  The redemption date is randomly selected within the overlap between the reward validity period, the member's join date, and the simulation period.

- **Birthday Rewards**
  Birthday rewards are generated separately from regular reward redemptions. For each member, a birthday reward is added if the member's birthday falls within the simulation period and occurs after the member's join date:

$$d_{\text{join},i} \leq d_{\text{birthday},iy} \leq d_{\text{end}}$$

  where:

  - $d_{\text{birthday},iy}$ is the birthday date of member i in year y.
  - $d_{\text{join},i}$ is the member's join date.
  - d_{end} is the simulation end date.

  Birthday rewards are recorded with zero point cost:

$$P_{\text{birthday}} = 0$$

  This allows the ledger to capture birthday benefit activity without reducing the member's point balance.

- **Running Balance**
  After all earn, expiry, redemption, and birthday events are generated, the records are sorted by member and event date. The running balance is computed cumulatively as:

$$B_{i,t} = \sum_{k=1}^{t} P_{i,k}$$

  where:

  - $B_{i,t}$ is the running point balance of member i after event t.
  - $P_{i,k}$ is the point change from event k for member i.

  This event-based structure allows the dataset to track how points are earned, redeemed, expired, and accumulated over time.

Output Datasets

	member_id	first_name	last_name	gender	email	dob	age	join_date	current_tier	tenure_months
0	1	Jack	English	Male	jackenglish24@gmail.com	1969-05-22	56	2025-06-23	Bronze	10
1	2	Jill	Byrd	Female	jillbyrd@yahoo.com	2006-10-10	19	2025-12-21	Bronze	4
2	3	Patty	Hughes	Female	pattyhughes@gmail.com	1994-07-21	31	2022-01-22	Bronze	51
3	4	Christina	Smith	Female	christinasmith@outlook.com	1975-11-11	50	2022-09-20	Bronze	43
4	5	Sharon	Berry	Female	sharonberry42@gmail.com	1999-07-14	26	2022-10-16	Bronze	43

Figure 2: Members

	transaction_id	member_id	transaction_date	amount	item_count	discount_amount
0	1	413	2022-01-10 20:05:05	125.0	1	0.0
1	2	753	2022-01-12 20:08:47	281.5	4	38.5
2	3	753	2022-01-22 20:38:40	102.5	1	17.5
3	4	1025	2022-01-24 03:11:14	379.0	4	241.0
4	5	764	2022-01-28 14:31:10	645.0	3	0.0

Figure 3: Transactions

	tier_id	tier_name	member_count	point_multiplier
0	1	Bronze	848	1.0
1	2	Silver	236	1.2
2	3	Gold	120	1.5

Figure 4: Tiers

	reward_id	reward_name	reward_type	min_membership_tier	points_cost	valid_start	valid_end
0	1	Free Brewed Coffee	Food	Bronze	150	2022-01-01	2022-03-20
1	2	Free Iced Coffee	Food	Bronze	280	2022-01-01	2022-03-20
2	3	Planner Set	Merch	Silver	1950	2022-01-01	2022-03-20
3	4	Free Pastry	Food	Bronze	250	2022-01-01	2022-03-20
4	5	Free Sticker Pack	Merch	Bronze	200	2022-01-01	2022-03-20

Figure 5: Rewards

	ledger_id	member_id	transaction_id	reward_id	event_type	point_type	points_change	event_date	running_balance
0	1	1	18968	None	EARN	TRANSACTION	320	2025-08-27 17:33:10	320
1	2	1	None	77	BURN	REDEEM	-150	2025-09-05 00:00:00	170
2	3	1	20803	None	EARN	TRANSACTION	300	2025-11-03 18:50:10	470
3	4	1	22474	None	EARN	TRANSACTION	700	2025-12-20 18:31:58	1170
4	5	3	32	None	EARN	TRANSACTION	330	2022-03-05 19:17:10	330

Figure 6: Points Ledger

Setup and Reproducibility

Environment Requirements

The data generation pipeline can be executed on a local workstation or a cloud-based environment (e.g. Google Colab, Kaggle Notebook). The project is implemented in Python and can be run using Jupyter Notebook or any Python-compatible IDE. Python version 3.9 or higher is recommended to ensure compatibility with all required libraries.

Dependencies

This project is dependent on the following Python packages:

Library	**Purpose**
pandas	Data manipulation and table generation
numPy	Numerical computations and random sampling
Faker	Synthetic generation of realistic member data
matplotlib	Data visualization for validation and EDA
seaborn	Enhanced statistical visualization
random	Built-in module for controlled randomness
datetime	Handling datetime data

Table 11: Dependencies

Cloud-based environments such as Google Colab already have common Python libraries (e.g. pandas, numpy) pre-installed.

To install unavailable libraries within the notebook cell, use:

```
!pip install package-name
```

If running locally, installation may also be done using the Windows command prompt (CMD):

```
pip --ver # check if pip is available
pip install package-name
```

Execution Instructions

To reproduce the dataset:
1. Open the notebook file using preferred environment (local or cloud).
2. Install the required dependencies (see Table 5).
3. Run all cells from top to bottom.
4. The generated datasets will be saved to the specified output directory.

Limitations and Future Improvements

- **Limited real-world validation.** The dataset was generated using synthetic rules and assumptions rather than actual customer records. While the logic was designed to reflect realistic loyalty program behavior, the generated patterns may not fully capture all variations found in real-world retail data. Future work may compare the simulated outputs with actual loyalty program datasets to refine the assumptions and improve realism.

- **Simplified customer behavior.** Customer activity was modeled using controlled probabilistic rules such as age-based transaction frequency, seasonality, and redemption probability. These assumptions are useful for simulation, but they do not account for more complex factors such as customer preferences, income level, product category, or location. Future versions may include additional behavioral variables to produce richer customer profiles.

- **Fixed tier and reward rules.** The tier assignment, point multipliers, reward costs, and expiration rules were predefined in the system. This makes the dataset easier to reproduce and interpret, but it may not reflect loyalty programs that frequently change their policies. Future improvements may allow dynamic tier thresholds, changing reward catalogs, and promotional campaigns.

- **Approximate seasonality.** Weekly and monthly seasonality were manually encoded using probability ranges and multipliers. This provides realistic temporal variation for simulation purposes, but it does not come from observed sales trends. Future work may calibrate these seasonal factors using real transaction data or industry benchmarks.

References

[1] Erika A. Lapitan. Market behavior for available coffee drinks in local coffee shops within los baños, laguna, 2022.

[2] Ma. Reina A. Mabeza. Pricing strategies and customer satisfaction in coffee shops: A case study in camarines norte, philippines. *International Journal of Multidisciplinary: Applied Business and Education Research*, 5(11):4284–4300, 2024.